Rice's Theorem

A Code-Based Perspective (Python)

Theory of Computation

1. Programs as Data

In Python, functions are first-class objects. We can pass a function's code into another function.

```
def program_a(x):
    return x * 2

def program_b(x):
    while True:
        pass # Infinite loop

# We want to write a "Static Analyzer" tool.
# It reads the source code and tells us something about it.
source_code = inspect.getsource(program_a)
```

2. What is a "Semantic Property"?

A property is just a Boolean question about what the code **does** (its semantics), not how it looks (its syntax).

```
# Example of a semantic property:
# "Does this function ever return 0?"

def returns_zero(source_code):
    # ??? How do we write this perfectly for ANY code ???
    pass

# We want:
# returns_zero(program_a) -> True (if input is 0)
# returns_zero(program_b) -> False (it never returns)
```

3. Trivial vs. Non-Trivial

Rice's theorem only applies to **non-trivial** properties.

```
# TRIVIAL: Applies to all programs (Boring, Decidable)
def is_a_program(source_code):
    return True

# TRIVIAL: Applies to no programs (Boring, Decidable)
def returns_a_live_unicorn(source_code):
    return False

# NON-TRIVIAL: Applies to some (Interesting, Undecidable!)
def prints_hello_world(source_code):
    # Some programs do, some don't.
    pass
```

4. The Dream: The Perfect Analyzer

Let's assume we are brilliant engineers and we wrote a perfect analyzer for a non-trivial property.

```
def does_it_return_zero(source_code):
    """
    Analyzes any source code perfectly.
    Returns True if the code can return 0.
    Returns False otherwise.
    Never gets stuck in an infinite loop itself.
    """
    # ... Magic ML / AI / Graph analysis logic ...
    return analysis_result
```

5. Setting the Trap (The Reduction)

If `does_it_return_zero` exists, we can use it to build a tool that solves the Halting Problem.

```
def halts_solver(target_source_code, input_val):
    """
    Tries to solve if target_source_code halts on input_val.
    """
    # We construct a malicious new program on the fly!
    evil_program = f"""
def wrapper_program():
    # 1. Run the target code
    {target_source_code}({input_val})

    # 2. If it ever finishes, return 0
    return 0
    """
```

6. Springing the Trap

Now we feed our `evil_program` to our "perfect" analyzer.

```
def halts_solver(target_source_code, input_val):
    # ... (evil_program created on previous slide) ...

    # If the wrapper returns 0, it means the target halted!
    # If it loops forever, it never reaches `return 0`.

    if does_it_return_zero(evil_program):
        return "YES, IT HALTS"
    else:
        return "NO, IT LOOPS FOREVER"
```

Wait! We just solved the Halting Problem. But Alan Turing proved mathematically in 1936 that `halts_solver` cannot exist.

7. The Contradiction

Because `halts_solver` cannot exist... our magic analyzer `does_it_return_zero` **cannot exist either**.

```
# Rice's Theorem mathematically guarantees that:
def analyze_any_non_trivial_behavior(code):
    raise UndecidableError("Impossible to compute!")

# This applies to:
# - Is there a buffer overflow?
# - Is this a virus?
# - Will this program crash?
```

8. What does this mean for Software Engineering?

The Reality of Static Analysis:

- Tools like `SonarQube`, `ESLint`, or Anti-Viruses cannot be 100% accurate.
- They **must** make compromises:
  - **False Positives:** Flagging safe code as an error.
  - **False Negatives:** Missing actual bugs or viruses.
- Rice's Theorem teaches us the absolute limits of automated code analysis.